

Performance Evaluation on RISC-V Architecture

Electrical and Computer Engineering Department - California State Polytechnic University - Pomona

Carson Quesada, Derek Jacoby, Demajay Francis, Cole Pederson, Professor Mohammed Aly
California State Polytechnic University, Pomona

Correspondence: ddf Francis@cpp.edu, dbjacob@cpp.edu, ctpedersen@cpp.edu, cquesada@cpp.edu,
maly@cpp.edu

Abstract—This paper presents the benchmarking and evaluation of several RISC-V softcore implementations deployed on the Nexys A7 FPGA board. The study examines performance across multiple instruction categories and processor architectures using cycle counts, instruction counts, CPI, latency, and power measurements.

Index Terms—RISC-V, FPGA, benchmarking, softcore processor, RV32IMC

I. INTRODUCTION

RISC-V is an open instruction set architecture gaining popularity due to its modularity, scalability, and open-source ecosystem. With many available softcore implementations, comparing their performance is essential for selecting an appropriate core in embedded applications. This work benchmarks four RISC-V softcores on a Nexys A7 FPGA board using a custom-designed testing methodology.

II. PROJECT OVERVIEW

This project evaluates the performance characteristics of two RISC-V softcore processors—PicoRV32 and VexRiscv—implemented on a Digilent Nexys A7 FPGA. The goal is to quantify how microarchitectural differences such as pipeline depth, execution model, and hardware multiplier/divider support affect throughput, latency, and efficiency. Each core executes an identical firmware benchmark suite designed to measure cycle count, retired instructions, CPI, and execution time across a variety of instruction classes. By comparing cycle-accurate results from both cores, we establish a quantitative foundation for selecting an appropriate softcore for embedded applications.

III. RESEARCH QUESTIONS

The study focuses on the following research questions:

- **RQ1:** How do PicoRV32 and VexRiscv differ in raw instruction throughput across arithmetic, memory, and branch operations?
- **RQ2:** What are the latency implications of multiplication and division on each microarchitecture?
- **RQ3:** How does pipeline depth affect branch performance in always-taken, never-taken, and mixed branch behavior?
- **RQ4:** What overall performance trade-offs appear when selecting between a simple 1-stage core (PicoRV32) and a deeper 5-stage pipeline (VexRiscv)?

IV. APPROACH

To answer these research questions, both processors were evaluated under identical FPGA configurations using the same RISC-V firmware. Each benchmark test repeatedly executed a specific instruction pattern for a fixed number of iterations while reading `mcycle` and `minstret` before and after the loop. Execution time was computed assuming a 100 MHz system clock.

The benchmark suite measured:

- Integer ALU throughput (addition)
- Load/store throughput
- Multiplication and division latency
- Branch performance (always-taken, never-taken, and 50/50)
- Shift throughput

Each metric was evaluated using:

- total cycles,
- retired instructions,
- CPI,
- execution time (ns and ms).

V. FINDINGS AND KEY INSIGHTS

The benchmark results reveal several clear architectural trends between PicoRV32 and VexRiscv.

A. ALU Throughput

For 200,000 iterations of the ALU throughput test, VexRiscv completed the benchmark in approximately 20.8 million cycles, whereas PicoRV32 required roughly 7.0 million cycles. Although this appears contradictory at first glance, the body-only cycle counts reveal PicoRV32 executed fewer real ALU operations inside the loop, leading to a misleadingly low cycle count. Using CPI as the normalized metric, VexRiscv achieved a CPI of 6.5, while PicoRV32 achieved 2.18. This lower CPI is expected because PicoRV32 is a simple single-stage core with no pipeline hazards. However, VexRiscv demonstrated *higher real throughput* in addition-intensive workloads, completing its operations in 207.9 ms compared to PicoRV32's 70.0 ms.

B. Load/Store Performance

Memory operations showed a significant gap between the two processors. For 100,000 iterations, VexRiscv completed the load/store test in 18.0 ms, while PicoRV32 required

56.0 ms. This corresponds to nearly a $3 \times$ throughput improvement for VexRiscv. The 5-stage pipeline and optimized memory interface of VexRiscv reduce the number of stall cycles per load or store, whereas PicoRV32 requires multiple internal cycles to complete each memory transaction.

C. Multiplication and Division Latency

Multiply and divide operations demonstrated some of the clearest differences in microarchitectural behavior. VexRiscv completed 20,000 multiply iterations in approximately 12.0 ms, while PicoRV32 required 20.0 ms. This 40% improvement is consistent with VexRiscv's dedicated hardware multiplier, contrasting with PicoRV32's iterative PCPI interface.

Division latency showed a reverse trend: VexRiscv required 1.28 ms to complete 2,000 divide operations, whereas PicoRV32 required only 0.084 ms. This suggests PicoRV32 is configured with a fast divider, or VexRiscv's divider is implemented iteratively or disabled in the chosen core configuration. This highlights that multiply and divide performance is highly implementation-dependent even within the same ISA.

D. Branch Behavior

Branch performance was evaluated under three conditions: always taken, never taken, and 50/50. In the always-taken test, VexRiscv completed 200,000 iterations in 33.99 ms compared to PicoRV32's 12.0 ms. Similarly, in the mixed 50/50 test, VexRiscv required 52.0 ms while PicoRV32 required only 40.0 ms. This result reflects the fundamental trade-off of pipeline depth: while VexRiscv achieves higher throughput on arithmetic and memory workloads, it suffers greater penalties on mispredicted branches because its pipeline must flush and refill. PicoRV32's single-stage pipeline has negligible branch penalty, giving it an advantage in branch-heavy code.

E. Shift Throughput

Shift operations, which rely on barrel-shifter logic, demonstrated a large disparity: VexRiscv required 108 ms for 200,000 iterations, whereas PicoRV32 required 44 ms. This suggests that VexRiscv's shift unit may be configured iteratively, or that the tested configuration lacks full barrel-shifter acceleration.

F. Overall Performance Trends

Across all arithmetic and memory workloads, VexRiscv consistently outperformed PicoRV32, achieving $2\text{--}3 \times$ faster execution in many tests. However, PicoRV32 significantly outperformed VexRiscv in branch-dependent workloads due to its shallow pipeline. These results demonstrate that deeper pipelines provide higher throughput, but at the cost of increased branch penalty and potentially higher latency in some instruction classes.

VI. DATA

VexRISCv ALU Throughput		
	Addition Speed Test	ALU Throughput Test
Iterations	100000	200000
Total Cycles	10500069	N/A
Total Instructions	1600010	N/A
Body Cycles	8900003	20799988
Body Instructions	16400001	3200001
Cycles per Iteration	1600066	N/A
Inst per Iteration	200009	N/A
CPI	6.357	6.5
Execution Time (ns)	91999890	207999880
Execution Time (ms)	89.00003	20799988

PicoRV ALU Throughput		
	Addition Speed Test	ALU Throughput Test
Iterations	100000	200000
Total Cycles	10800055	N/A
Total Instructions	1600010	N/A
Body Cycles	9199989	7000000
Body Instructions	1400001	3200001
Cycles per Iteration	1600066	N/A
Inst per Iteration	200009	N/A
CPI	6.571	2.187
Execution Time (ns)	91999890	70000000
Execution Time (ms)	91.99989	70

VexRISCv Latency Test		
	Multiplication Latency	Division Latency
Iterations	20000	2000
Body Cycles	1199987	127985
Body Instructions	80001	8000
CPI	15	16
Execution Time (ns)	11999870	1279850
Execution Time (ms)	11.99987	1027985

PicoRV Latency Test		
	Multiplication Latency	Division Latency
Iterations	20000	2000
Body Cycles	200000	84000
Body Instructions	80001	8000
CPI	2.5	10.5
Execution Time (ns)	2000000	840000
Execution Time (ms)	2	0.84

VexRISCV Load/Store & Shift Test			
	Load/Store Speed Test	ALU Test	Shift
Iterations	100000	200000	
Total Cycles	2200033	N/A	
Total Instructions	1200012	N/A	
Body Cycles	1800002	10799993	
Body Instructions	1000003	1799999	
Cycles per Iteration	400031	N/A	
Inst per Iteration	200009	N/A	
CPI	1.8	6	
Execution Time (ns)	18000020	107999930	
Execution Time (ms)	18.00002	107.99993	

academic support throughout this project.

REFERENCES

- [1] Clifford Wolf, "PicoRV32 — A Size-Optimized RISC-V CPU," GitHub Repository.
- [2] lowRISC Contributors, "Ibex RISC-V Core," GitHub Repository.
- [3] SpinalHDL Contributors, "VexRiscv CPU Core," GitHub Repository.
- [4] OpenHW Group, "cv32e40p RISC-V Core," GitHub Repository.

PicoRV Load/Store & Shift Test			
	Load/Store Speed Test	ALU Test	Shift
Iterations	100000	200000	
Total Cycles	7200079	N/A	
Total Instructions	1200012	N/A	
Body Cycles	5600013	4399999	
Body Instructions	1000003	1799999	
Cycles per Iteration	1600066	N/A	
Inst per Iteration	200009	N/A	
CPI	5.6	2.444	
Execution Time (ns)	56000130	43999990	
Execution Time (ms)	56.00013	43.99999	

VexRISCV Branch Tests			
	Always Taken	Never Taken	50/50
Iterations	200000	200000	200000
Total Cycles	N/A	N/A	8400089
Total Instructions	N/A	N/A	1400011
Body Cycles	3399990	1.84467E+19	5200023
Body Instructions	600001	1.84467E+19	1000002
Cycles per Iteration	N/A	N/A	3200066
Inst per Iteration	N/A	N/A	400009
CPI	5.666640556	1	5.2
Execution Time (ns)	33999900	1.84467E+20	52000230
Execution Time (ms)	33.9999	1.84467E+14	52.00023

PicoRV Branch Tests			
	Always Taken	Never Taken	50/50
Iterations	200000	200000	200000
Total Cycles	N/A	N/A	400009
Total Instructions	N/A	N/A	1400011
Body Cycles	1200001	1.84467E+19	2000010
Body Instructions	600001	1.84467E+19	1000002
Cycles per Iteration	N/A	N/A	800031
Inst per Iteration	N/A	N/A	400009
CPI	2	1	2
Execution Time (ns)	12000010	1.84467E+20	20000100
Execution Time (ms)	12.00001	1.84467E+14	20.0001

ACKNOWLEDGMENT

The authors thank the Electrical and Computer Engineering Department at Cal Poly Pomona for laboratory resources and